Reviewer Pack — Minimal Order of p-Adic Galois Representations and Communica…

Entry 91276b734435 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 08:32:31 UTC

Minimal Order of p-Adic Galois Representations and Communication Rank Growth

Entry ID: `91276b734435` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 08:32:31 UTC

1. Conjecture

Field A (mathematical branch): p-adic Galois Theory **Field B** (complexity object): Communication Complexity

Statement:

> The minimal order of p-adic Galois representations attached to a given boolean function f is linearly correlated with the communication rank growth of f, such that log(ρ(f)) = Θ(rank_gal(f)), where ρ(f) denotes the minimal order of the p-adic Galois representation and rank_gal(f) is the communication rank of f.

Rationale (proposer's reasoning):

> p-adic Galois theory provides a framework for studying arithmetic properties of boolean functions. The connection between these arithmetic properties and communication complexity could reveal new insights into the inherent structure of computational tasks, potentially leading to novel hardness results or algorithmic improvements.

Taxonomy category: `p-adic-Galois-Theory × Communication Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `daef441b80e2d2fb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated boolean functions f with up to 40 variables, the correlation coefficient between log(ρ(f)) and rank_gal(f) is significantly positive (p-value < 0.05), where ρ(f) is the minimal order of the p-adic Galois representation and rank_gal(f) is the communication rank of f.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_boolean_function(n: int) -> list:
    return [random.choice([0, 1]) for _ in range(2**n)]

def communication_rank(f: list) -> int:
    n = len(f)
    if n == 1:
        return 1
    rank = 1
    for i in range(n):
        if f[i] != f[0]:
            break
    else:
        return rank
    for j in range(1, n):
        if f[j] != f[1]:
            break
    else:
        return rank
    rank += 1
    for k in range(2, n):
        if f[k] != f[2]:
            break
    else:
        return rank
    rank += 1
    return rank

def p_adic_galois_representation(f: list) -> int:
    n = len(f)
```

```
    rho = 1
    for i in range(n):
        if any(f[i * 2**(n-j-1): (i+1) * 2**(n-j-1)] == f[(i+1) * 2**(n-j-1): (i+2) * 2**(n-j-1)]
            rho += 1
    return rho

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    f = generate_boolean_function(n)
    rho = p_adic_galois_representation(f)
    rank_gal = communication_rank(f)
    metric_value = math.log(rho)
    instances_tested = 1
    n_max = n
    conjecture_holds = False
    counterexample = ""

    return {
        "metric_name": "log_rho",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = f"seed={result['seed']}, log_rho={result['metric_value']}"
                print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={
                break
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13669
2	propose	ollama_remote	glm4:latest	0	0	15818
3	preregistration	ollama_remote	glm4:latest	0	0	14463
4	novelty	ollama_remote	glm4:latest	0	0	18177
5	novelty	ollama_remote	glm4:latest	0	0	8764
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20198
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13438
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9764
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37689
10	judge	ollama_remote	glm4:latest	0	0	42682

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 194662 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/91276b734435.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/91276b734435.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/91276b734435.tar.gz` (if generated)